Using Helgrind to detect threading issues

Let us take an example of a multithreaded C program which uses POSIX pthread APIs for synchronisation. The program compiles and runs without any warnings/errors. We will analyse the program using Helgrind to detect any thread synchronisation issues.

```
 1 /* Program race.c */
 2
 3 #include <stdio.h>
 4 #include <pthread.h>
 5
 6 int shared_counter = 0;
 7 pthread_mutex_t mutex;
 8
 9 void *inc_counter(void *arg) {
10     int current_value;
11     int i;
12     for (i = 0; i < 10000; i++) {
13
14         pthread_mutex_lock(&mutex);
15         // Read the shared variable
16         current_value = shared_counter;
17         pthread_mutex_unlock(&mutex);
18
19         // Modify the local copy
20         current_value++;
21
22         // Write back to the shared variable
23         shared_counter = current_value;
24     }
25     return NULL;
26 }
27
28 int main() {
29     pthread_t thread1, thread2;
30
31     pthread_mutex_init(&mutex, NULL);
32
33     pthread_create(&thread1, NULL, inc_counter, NULL);
34     pthread_create(&thread2, NULL, inc_counter, NULL);
35
36     pthread_join(thread1, NULL);
37     pthread_join(thread2, NULL);
38
39     pthread_mutex_destroy(&mutex);
40
41     printf("Final value of shared counter: %d\n", shared_
counter);

42     return 0;
43 }
```

This program (race.c) creates three threads – main() thread, thread1 and thread2. Thread1 and thread2 increment the counter (shared_counter variable) 10,000 times. We will use Helgrind to detect any thread synchronisation issue.

First, compile the program as follows:

```
$ gcc -g -o race race.c -lpthread
```

Then run the program using Helgrind:

```
$ valgrind --tool=helgrind ./race

==16932== Helgrind, a thread error detector
==16932== Copyright (C) 2007-2013, and GNU GPL'd, by
OpenWorks LLP et al.
==16932== Using Valgrind-3.10.0 and LibVEX; rerun with -h for
copyright info
==16932== Command: ./race
==16932==
==16932== ---Thread-Announceme
nt------------------------------------------
==16932==
==16932== Thread #3 was created
==16932==    at 0x514D1DE: clone (in /usr/lib64/libc-2.17.so)
<Snipped>
==16932==    by 0x514D21C: clone (in /usr/lib64/libc-2.17.so)
==16932==  Address 0x601084 is 0 bytes inside data symbol
"shared_counter"
==16932==
Final value of shared counter: 20000
==16932==
==16932== For counts of detected and suppressed errors, rerun
with: -v
==16932== Use --history-level=approx or =none to gain
increased speed, at
==16932== the cost of reduced accuracy of conflicting-access
information
==16932== ERROR SUMMARY: 20000 errors from 2 contexts
(suppressed: 30005 from 8)
```

The output of Helgrind suggests some race conditions. Let's interpret it.

1. In the Helgrind output, the two threads are indicated by Thread #2 and Thread #3 (main() is the first thread).
2. The output suggests two race scenarios.
3. The first race situation is:

```
==16932== Possible data race during read of size 4 at
0x601084 by thread #3
==16932== Locks held: 1, at address 0x6010A0
==16932==    at 0x40080F: inc_counter (race.c:16)
==16932==    by 0x4C2F881: ??? (in /usr/lib64/valgrind/
```